

Randomness, Entropy, IID, and Structure: Understanding DRBGs and Statistical Tests

shahiN Noursalehi
(mixoftix@gmail.com)

Introduction

In cryptography, randomness is not a single concept. Two generators may produce outputs that look similar, yet behave very differently under statistical tests. This document explains the relationship between entropy, deterministic random bit generators (DRBGs), IID assumptions, and randomness tests such as NIST SP 800-22's Discrete Fourier Transform (DFT) test.

It also clarifies why a generator can pass NIST SP 800-90B (entropy validation) but fail DFT, and why this does not necessarily indicate insecurity.

1. Entropy vs Randomness

Entropy

Entropy measures unpredictability. If an attacker cannot guess the next bit, the entropy is high.

NIST SP 800-90B focuses on:

- noise source quality
- min-entropy
- IID vs non-IID behavior
- restart tests
- conditioning functions

Passing 800-90B means the entropy source is fundamentally sound.

Randomness Tests

Randomness tests measure patternlessness, not unpredictability. They detect:

- periodicity
- cycles
- structural artifacts
- correlations
- spectral spikes

Passing 800-22 means the output resembles IID noise.

These two concepts are different. A generator can be unpredictable (high entropy) but still contain detectable structure.

2. What a DRBG Is

A Deterministic Random Bit Generator (DRBG) expands a seed into a long stream of pseudorandom bits.

Properties:

- deterministic
- secure if the seed is unpredictable
- may fail statistical tests
- widely used in cryptography

Examples include AES-CTR-DRBG, Hash-DRBG, HMAC-DRBG, ChaCha20-based DRBGs, Windows CNG, and OpenSSL FIPS DRBG.

These DRBGs are secure even though they often fail DFT due to internal structure.

3. IID: Independent and Identically Distributed

Many randomness tests assume the data is IID.

Independent

Each sample does not depend on previous samples. No memory, no correlation, no influence from earlier values.

Identically Distributed

Each sample comes from the same probability distribution. No drift, no bias changes, no evolving behavior.

Why IID matters

Most statistical tests in NIST SP 800-22 assume the input is IID. If a generator is deterministic, periodic, stateful, conditioned, or has internal structure, then it is not IID—even if it is cryptographically secure.

This is why:

- DRBGs often fail DFT
- entropy sources pass 800-90B but fail 800-22
- sine-based noise fails DFT
- hash-mixed noise passes DFT

DFT specifically looks for non-IID behavior, especially periodicity.

Key point

A generator can be non-IID and still be cryptographically secure.

4. Why DFT Fails Even When Entropy Is Good

The Discrete Fourier Transform test detects periodicity.

A DFT failure means:

- the sequence contains repeating motifs
- there is a detectable frequency spike
- the output is not IID noise

It does not mean:

- entropy is low
- the generator is insecure
- the DRBG is broken

DFT is extremely sensitive and often flags harmless structure.

5. The “Tone in the Noise” Problem

If a randomness source includes a periodic component, DFT will detect it.

Example of a problematic noise function:

```
t = (timerMs * 0.001) + noisyCallCounter * 0.0001
noise = sin(t * 12.9898 + t * t * 0.125) % 1
```

This produces:

- sine waves
- quadratic chirps
- harmonics
- repeating cycles

These create spectral spikes, which DFT is designed to catch.

Even if XORed with cryptographic randomness, periodic structure can leak through.

6. Deterministic Does Not Mean Insecure

A deterministic generator can be secure, unpredictable, and cryptographically strong, yet still fail statistical tests.

Cryptographic security depends on unpredictability, not on looking like white noise.

NIST SP 800-90B measures unpredictability. NIST SP 800-22 measures patternlessness.

They answer different questions.

7. When a DFT Failure Matters

A DFT failure is meaningful only if:

- it happens consistently
- it appears across many independent samples
- it correlates with other structural tests
- it indicates a real periodic artifact in the entropy source

A single DFT failure is normal. Even a few failures are expected at the test's significance level.

If a generator passes 800-90B but occasionally fails DFT, the entropy is good, but the output has mild structure. This is not a security problem.

8. Eliminating Periodicity

To remove periodic artifacts, avoid sine-based noise and float operations.

Use a **non-periodic integer mixer** instead:

```
z = mix32(timerMs ^ noisyCallCounter)
noise = mix32(z + 0x9e3779b9)
```

With:

```
function mix32(x) {
  x ^= x >>> 16
  x = Math.imul(x, 0x7feb352d)
  x ^= x >>> 15
  x = Math.imul(x, 0x846ca68b)
  x ^= x >>> 16
  return x >>> 0
}
```

This produces avalanche mixing, no periodicity, no float artifacts, no spectral spikes, and DFT-safe output. This approach is used in modern PRNGs and entropy conditioners.

9. Key Takeaways

- Entropy $\neq$ Randomness Entropy measures unpredictability; randomness tests measure patternlessness.
- DRBGs are deterministic They can be secure even if they fail statistical tests.
- IID matters for statistical tests Many tests assume IID input; DRBGs often violate this assumption.
- DFT detects periodic structure A failure means “tone in the noise,” not “bad entropy.”
- Passing 800-90B is more important It validates the entropy source.
- Sine-based noise creates periodicity Avoid float-based chaotic functions.
- Avalanche mixers remove structure Use integer hashing instead of trigonometric noise.
- Rare DFT failures are normal Only systematic failures indicate a real issue.

Conclusion

Randomness is multi-layered. Cryptographic security depends on entropy and unpredictability, not on passing every statistical test. A generator can pass NIST SP 800-90B and still fail DFT without being insecure. Understanding the difference between entropy, deterministic structure, IID assumptions, and statistical randomness is essential for anyone working with cryptographic systems.

Acknowledgement

This article was developed with the assistance of **Microsoft Copilot**, whose explanations, structure, and clarity helped refine the concepts and make them accessible for practitioners.